RESEARCH · OCT 15, 2026 · PART TWO

Governance on a database

If a company's model lived in a database instead of Markdown in git on GitHub, **what would its tooling have to do** so that every check still holds? Forty requirements, each tied to **what git or GitHub does today** and **the checks that rest on it**.

The levels

Each level adds one thing that reads from the model.

5	**Published**	pages drawn from the model at a pinned state; the editor reads the same model	3 requirements
4	**Served**	a read-only server and a chat answer from one named state	4 requirements
3	**Governed**	one form, pins, seat-authored changes, rules and controls as pages	16 requirements
2	**Gated**	the same check on every change before it reaches the main state	14 requirements
1	**Model**	the model is a folder of Markdown, with the check, the skills and a commit hook	3 requirements

What git and GitHub do that nobody wrote down

Most of the governance found in the first part is not in CompanyGraph's own code. The checker reads files; everything that makes its verdict binding comes from git and GitHub: a state of the whole model that has a name and never changes, a change made apart from that state and applied to it whole, an author kept with each change, a gate that refuses to apply a change its checks have not passed, an account that alone may apply it, and a record of every time someone went past the gate. A database setup has to supply each of those itself, or the checks still run and govern nothing.

The requirements fall into ten groups. A requirement is written as the tooling must do it, then what does it today, then the checks and controls of the three models that rest on it, the level of the talk it serves, and how anyone can tell it is met.

A. States and history

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-01	give every applied state of the whole model an id that names exactly that content and never another	a git commit and its hash	the pins in `source.json`; the served-commit check; the commit every MCP answer names under `model`; `build:check`, `model:check` and `pages:check`	3	two reads of one id return the same bytes, always; a changed byte is a new id

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-02	read any past state whole, as of its id	`git checkout`, a clone at a commit	a site and a server drawing the model at a pinned state; the change checks reading a proposal's base	4	a state from a year ago is read and checks as it did then
GD-03	compute the difference between two states per entity, per field and per section, pairing an entity across a rename by its id	`git diff`, rename detection	the change checks (an id never changes, a decision is not rewritten, a label stays with its item); `design german stale`; the Reviewer reading the diff	2	renaming an entity shows as a rename, not as a deletion and a new page
GD-04	keep history append-only: nothing on the main state is rewritten or deleted, but for a purge of named content (GD-30)	the ruleset's refusal of a force push and of a deletion	control Main takes a change only through a green, current pull request; rule What is cited from outside the model is not rewritten	2	an attempt to rewrite or delete a past state is refused and recorded
GD-05	record for every change its author, the person accountable for it, the time, and structured fields for its process, phase and track, and the tool that co-wrote it	the commit's author, committer, date and trailers	the seat check; rule A commit keeps the author it was made with; decision An agent's commit is authored by the seat it held	3	a change applied without a process, a phase or an author is refused

B. Proposals

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-06	let a change be prepared apart from the main state, as a named change set holding edits to any number of entities, several open at once and none visible to readers of the main state	a branch, a worktree beside the clone	every check that runs on a pull request; working in parallel sessions	2	two open change sets touch the same entity and neither sees the other until it is applied
GD-07	apply a change set whole or not at all, as one new state	a merge commit	the instance check holding the whole model, not a half-applied one	2	a change set that fails halfway leaves the main state as it was
GD-08	apply a change set only on top of the current main state, so one based on an older state is brought up to date and checked again first	required checks in strict mode; merging main into the branch	rule A state is read again before it is relied on	2	a change set whose base moved cannot be applied until it is re-checked on the new base
GD-09	detect two change sets editing the same entity and make the second resolve it, never overwrite it	a merge conflict	every check; nothing is lost silently	2	the second of two edits to one field is refused until resolved
GD-10	keep authorship through applying: the change keeps its author, and applying adds who applied it	merge commit only, squash and rebase off	rule A commit keeps the author it was made with	3	an applied change still shows the seat that made it
GD-11	keep with each change set its record: the difference, the discussion, each check's result and output, and who applied it and when	a pull request	control A reviewer reads the output, not the report; KPI Merges Past Their Checks	2	a year later, any applied change shows what was checked and who applied it

C. Checks

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-12	run the checker on every change set and on every applied state, and refuse to apply a change set until each named check has passed	required status checks; the `companygraph` job	the instance checks; feature Checks an instance runs; use case Check a change against the schemas	2	a change set whose required check failed or never ran cannot be applied

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-13	run, for each instance, the checker release the instance adopted, and refuse a run by any other	the manifest's `tooling` and the workflow's ref	the tooling pin guard; requirement An instance is checked only against the release it adopted	2	a newer checker refuses an instance that has not taken it
GD-14	give a check the whole proposed state and its base, so a check can hold the change and not only its result	`ids --range` over base..head	the change checks; the seat check over a range	2	the id check refuses a change set that alters an id, and passes one that does not
GD-15	keep each check's output, link it from the change set, and tell a failure from a note and both from what was not checked	the check run's log and conclusion	rule A check counts only when its output was seen; requirement A check that passes says what it left	2	the output of any past check is readable, with its notes and what it did not check
GD-16	check a change while it is written, with the same checker and the same schemas as the gate	the editor plugin; the git hooks	feature Checks while a file is edited; the edit lock on headings and ids	5	an editor shows the same failure the gate would, before the change is proposed
GD-17	refuse a change set that names no seat its phase lists, when it is created and again when it is applied	the `commit-msg` hook and the seat check in CI	decision An agent's commit is authored by the seat it held	3	a change set authored by a seat the phase does not list never reaches the main state
GD-18	store one canonical form, so that two writers making the same change store the same state	the one Markdown form, `companygraph form`	feature A repository is held to one form and told which pins are behind	1	the same edit made in two editors produces no difference

D. Who may do what

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-19	change the main state only by applying a change set, never by a direct write	the ruleset's pull request rule	control Main takes a change only through a green, current pull request	2	a direct write to the main state is refused, for every account
GD-20	restrict who may apply to named principals, while others may propose	the push restriction to the Owner's account	control Only the Owner's account changes main; rule The Owner's word merges	3	an account that may propose is refused when it applies
GD-21	let only named principals go past a gate, and record every time one does with what it skipped	the ruleset's admin bypass and GitHub's rule suites	KPI Merges Past Their Checks	2	each bypass in a week can be listed with the check it skipped
GD-22	let an agent act under rights narrower than its operator's: propose but not apply, unless the Owner gives the word for that change in that conversation	the agent harness's permission layer	control An agent's permissions refuse a merge it was not given; control A message from another session carries no word of the Owner's	3	an agent's apply without the Owner's word is refused, whatever another agent asks
GD-23	set who may read each instance, public for a published model and private for another	repository visibility and the organization's base permission	the read-only server; the sites; a private instance	4	an instance set private is not readable by an account outside it

E. Releases, adoption and pins

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-24	release schemas, core and each pack, as named versions with notes, which nobody but the Owner can create, move or delete	a git tag and a GitHub Release; the release-tag ruleset	control A release tag stays where the Owner put it; decision A release is its tag	3	an attempt to move a released version is refused
GD-25	let an instance adopt a schema version on its own decision, hold a copy it can prove unaltered, and move only when it decides to	core vendored at a release with a hash per file in the manifest	the vendored file hashes; control A vendored copy is held to the release it names; decision Core is vendored into an instance at a release its manifest names	2	a schema altered inside an instance fails its check; a new release changes no instance until it adopts it
GD-26	let a consumer pin what it takes, a schema version or a model state, by id, in a line anyone can read, and report which pins are behind	`pins.json`, `source.json`, `conventions.json`; `companygraph pins`; the family report	concept Pin; the family report; `pin:check`	3	each consumer's pins and how far behind they are can be listed
GD-27	let one run move many pins at a time while refusing a second run beside it	the family resync's lock and its refusal beside an open resync	control A family resync refuses to start beside another; rule One session holds a piece of shared work	3	a second run started beside the first is refused

F. Content safety

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-28	scan every change for a secret before it is stored, refuse one it finds, and record any push past it with its reason	secret scanning with push protection	control A secret is refused at the push; rule A secret is never written into a repository	3	a known token in a change is refused, and a bypass shows who and why
GD-29	watch the tooling's own dependencies for known vulnerabilities	Dependabot alerts and security updates	—	3	an advisory against a dependency raises an alert
GD-30	purge named content from every state that holds it, a guest's data or a secret, leaving a record of who purged what kind of content and when, and the ids of the states it touched	deleting from an issue's edit history; rotating a secret	control A guest's data in a request is taken out of it	3	purged content is unreadable in every past state, and the purge itself is listed

GD-30 is the one place a database setup must be stronger than git: git cannot take content out of a commit without rewriting every state after it, which GD-04 forbids, so the family keeps personal data and secrets out of the model altogether. A database that holds a company's operations will hold personal data, and needs a purge that GD-01 and GD-04 survive.

G. Publishing and serving

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-31	have every artifact published from the model, a page, a site, a server's snapshot, name the state it was built from, and check that it equals what that state builds	`build:check`, `model:check`, `pages:check`; the served-commit check	control Pages drawn from the model are held to it; control A deployed server is asked which commit it serves; rule The model is corrected first	5	a page edited by hand, or built from another state, fails its check

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-32	serve reads at one named state and say which state every answer comes from	the snapshot the MCP server takes at a commit	decision The server only reads, and answers at one named commit; concept-design Provenance	4	two answers given at one state agree, and each names that state
GD-33	publish to an outside registry only from an approved release, with a key no other step can reach	the `registry` environment	control A registry entry waits for a release tag and the Owner	4	a publish from anything but an approved release fails for want of the key
GD-34	deploy a published site only from the main state	the `github-pages` environment	—	5	a deploy from a change set is refused

H. Audit and measurement

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-35	keep an audit log of applies, bypasses, changes of rights and purges, which a scheduled job can read by week	GitHub's audit log and rule suites, read by the `kpi` workflow	KPI Merges Past Their Checks; KPI Change Fail Rate	3	the week's bypasses are counted from the log alone
GD-36	report on a schedule which pins are behind, what is unreleased, and which checks fail on the main state	the family report's issue	the family report	3	the report lists a failing check on the main state the day it fails

I. Requests and parallel work

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-37	take requests from outside as items anyone can open and comment on, with an edit history, linkable from the change that answers them	GitHub issues	process Feature request; control A request is asked for its gap and never its company	3	a request, its restatement and the change that answered it link to each other
GD-38	let every session and agent see the open change sets, who holds them and what they claim, before it starts shared work	the list of open pull requests, branches and tags	rule One session holds a piece of shared work	3	a session can list every open change set touching what it is about to change

J. Leaving the database

ID	THE TOOLING MUST	TODAY	WHAT RESTS ON IT	LEVEL	HOW TO TELL IT IS MET
GD-39	export any state to the Markdown form, ids kept, that the checker and every agent read today, and import it back unchanged	the model is Markdown	objective A company's model belongs to no tool, and any graph database can hold it; decision The schema written as prose is the only schema	1	a state exported and imported again is the same state, and the exported files pass `companygraph check`
GD-40	let an agent read and propose through the same skills it uses on files, or through tools that give it the same	the skills read files in a clone	the agent pass (`companygraph-validate`); every procedure skill	1	`companygraph-validate` runs against a state of the database

Where a database is harder than git

- **Change sets over a graph (GD-06, GD-07, GD-09).** Branching and merging are what git is for. A relational or graph database gives transactions, which are short; it has no long-lived, named, parallel change sets, and pairing edits to one entity in two of them is a problem the tooling has to solve, not inherit.
- **A state with a name (GD-01, GD-02).** Point-in-time reads are common; a content hash that names a whole state, so a pin means the same bytes forever, is not.
- **Differences by entity (GD-03).** A row-level difference is not the difference the change checks read: they compare a page's id, a decision's text and a label's item after both sides are put in one form.
- **Checking while writing, offline (GD-16).** Today the checker runs on a laptop with no network. A database setup runs it where the data is, so an editor needs either a local copy or a round trip per keystroke.
- **The purge (GD-30)** has no equivalent in git, and is where a database is asked to do more, not the same.
- **Agents (GD-40).** Every skill an instance ships reads files. A database setup either exports to files for them, or gives them tools as good as reading the model whole.

The setup the family runs today

The MCP server already keeps the Markdown in git as the record and loads each applied state into a snapshot it reads from, the snapshot it names under `model` in every answer. Read that way, git and GitHub meet every requirement in A to E as they do now, and the snapshot serves the reads of GD-32: a database put in front of git, in the place of that snapshot, would leave the governance where it is.

What the model took from it

The Owner made the requirements explicit in this model on October 15, 2026: thirty of them are requirement pages of CompanyGraph Core, each carrying its label here as `source-id`. GD-13 and GD-32 were already pages of the model, under other names. GD-27, GD-29, GD-33, GD-34, GD-36, GD-37 and GD-38 bind the company's own conduct, which its rules and controls hold, and are not requirements of the product.

The two questions this research left open were settled the same day. Personal data stays out of every model by rule, so no model needs a purge, and GD-30 was dropped. And the setup the family runs today is the supported one, by decision: a model is kept in git, and a store for reads stands in front of it; any other setup, or another git host, is supported once it shows it meets these requirements.

Sources

- The first part, `2026-10-14-governance-as-data.md` beside this file, and every source it names.
- The three models' controls, rules, decisions, KPIs and requirements, read on 2026-10-15.